Anexo 14 — Servidor web (FastAPI)

Archivo: Scripts/web_remote/server.py

Para que sirvio:
Backend de la interfaz. Expone el WebSocket de objetivo/telemetria, el stream de video MJPEG y los endpoints de acciones (agarrar, saludar, camara, apariencia).

```
"""FastAPI app que sirve el HTML del cliente y un WebSocket para comandar el target.

Diseñado para ser corrido en un thread daemon desde run_interactive.py.
Comunica con el sim loop principal via queue.Queue (thread-safe).

Cliente -> servidor: JSON {x, y, z} (posicion del target en metros, world frame).
Servidor -> cliente: JSON {theta_deg, focus, fps} (telemetria del estado del robot).
"""

import asyncio
import json
import queue
from pathlib import Path

from fastapi import FastAPI, WebSocket, WebSocketDisconnect
from fastapi.responses import FileResponse, StreamingResponse
from fastapi.staticfiles import StaticFiles


# Colas compartidas con el sim loop. Se inyectan desde fuera via init_queues().
_target_queue: queue.Queue | None = None
_telemetry_queue: queue.Queue | None = None
_frame_queue: queue.Queue | None = None
_event_queue: queue.Queue | None = None  # v13+: trigger discrete actions (grab_yellow)


def init_queues(target_q: queue.Queue, telemetry_q: queue.Queue,
                frame_q: queue.Queue | None = None,
                event_q: queue.Queue | None = None):
    """Llamar UNA vez desde el main loop antes de servir requests.

    `frame_q` es opcional para mantener compatibilidad.
    `event_q` (v13+): cola de eventos discretos del usuario (ej. boton "agarrar").
    """
    global _target_queue, _telemetry_queue, _frame_queue, _event_queue
    _target_queue = target_q
    _telemetry_queue = telemetry_q
    _frame_queue = frame_q
    _event_queue = event_q


STATIC_DIR = Path(__file__).parent / "static"
app = FastAPI(title="DUM remote", version="1.0")
app.mount("/static", StaticFiles(directory=STATIC_DIR), name="static")


@app.get("/")
async def index():
    """Sirve el cliente HTML."""
    return FileResponse(STATIC_DIR / "index.html")


@app.websocket("/ws")
async def ws(websocket: WebSocket):
    """WebSocket bidireccional. Cliente arrastra target -> sim lo aplica.
    Sim envia telemetria (theta, focus) -> cliente la muestra."""
    await websocket.accept()
    print("[WS] cliente conectado")

    async def receive_target():
        while True:
            try:
                msg = await websocket.receive_text()
            except WebSocketDisconnect:
                return
            try:
                data = json.loads(msg)
                if _target_queue is not None:
                    # Descarta target anterior, mantiene solo el ultimo
                    try:
                        _target_queue.get_nowait()
                    except queue.Empty:
                        pass
                    _target_queue.put_nowait(data)
```

```
            except (json.JSONDecodeError, queue.Full):
                pass

    async def send_telemetry():
        while True:
            await asyncio.sleep(0.05)  # 20 Hz de telemetria al cliente
            if _telemetry_queue is None:
                continue
            try:
                telem = _telemetry_queue.get_nowait()
            except queue.Empty:
                continue
            try:
                await websocket.send_json(telem)
            except WebSocketDisconnect:
                return

    try:
        await asyncio.gather(receive_target(), send_telemetry())
    except Exception as e:
        print(f"[WS] error: {e}")
    finally:
        print("[WS] cliente desconectado")


# --- MJPEG stream del render de MuJoCo -----------------------------------
# Boundary fijo para el multipart/x-mixed-replace; cualquier string vale, pero
# tiene que coincidir con el "Content-Type" de la respuesta y los separadores.
_MJPEG_BOUNDARY = "dumframe"


async def _mjpeg_generator():
    """Async generator que toma JPEG bytes de la cola y los emite como multipart.
    Si la cola esta vacia, hace sleep corto en vez de bloquear el event loop.
    Reutiliza el ultimo frame conocido para mantener visible la imagen si el
    sim loop momentaneamente no produce frames nuevos."""
    last_jpeg: bytes | None = None
    boundary_bytes = f"--{_MJPEG_BOUNDARY}\r\n".encode("ascii")
    while True:
        jpeg: bytes | None = None
        if _frame_queue is not None:
            try:
                jpeg = _frame_queue.get_nowait()
            except queue.Empty:
                jpeg = None
        if jpeg is None:
            if last_jpeg is None:
                # Aun no llego ningun frame: esperar un poco mas.
                await asyncio.sleep(0.05)
                continue
            jpeg = last_jpeg
        last_jpeg = jpeg
        chunk = (
            boundary_bytes
            + b"Content-Type: image/jpeg\r\n"
            + f"Content-Length: {len(jpeg)}\r\n\r\n".encode("ascii")
            + jpeg
            + b"\r\n"
        )
        try:
            yield chunk
        except (asyncio.CancelledError, GeneratorExit):
            return
        # ~30 fps techo del stream; el sim loop tipicamente produce menos.
        await asyncio.sleep(1 / 30)


@app.post("/grab_yellow")
async def grab_yellow():
    """Dispara el ciclo grab+throw del motor de animacion."""
    if _event_queue is None:
        from fastapi import Response
        return Response(content="event queue not initialized", status_code=503,
                        media_type="text/plain")
    try:
        _event_queue.put_nowait({"type": "grab_yellow"})
        return {"status": "queued", "event": "grab_yellow"}
    except queue.Full:
        return {"status": "queue_full"}


@app.post("/wave")
async def wave():
    """Dispara el saludo procedural (brazo random L/R)."""
    if _event_queue is None:
```

```
        from fastapi import Response
        return Response(content="event queue not initialized", status_code=503,
                        media_type="text/plain")
    try:
        _event_queue.put_nowait({"type": "wave"})
        return {"status": "queued", "event": "wave"}
    except queue.Full:
        return {"status": "queue_full"}


@app.post("/skin/{name}")
async def set_skin(name: str):
    """Cambia la apariencia del robot en tiempo real (ej. realista / imperio)."""
    if _event_queue is None:
        from fastapi import Response
        return Response(content="event queue not initialized", status_code=503,
                        media_type="text/plain")
    try:
        _event_queue.put_nowait({"type": "set_skin", "name": name})
        return {"status": "queued", "event": "set_skin", "name": name}
    except queue.Full:
        return {"status": "queue_full"}


@app.post("/cam/{direction}")
async def cam(direction: str):
    """Cambia la vista de la camara del stream. direction: 'next' o 'prev'."""
    if _event_queue is None:
        from fastapi import Response
        return Response(content="event queue not initialized", status_code=503,
                        media_type="text/plain")
    try:
        _event_queue.put_nowait({"type": "cam", "dir": direction})
        return {"status": "queued", "event": "cam", "dir": direction}
    except queue.Full:
        return {"status": "queue_full"}


@app.get("/stream")
async def stream():
    """MJPEG stream del viewer de MuJoCo. Pensado para `<img src="/stream">`."""
    if _frame_queue is None:
        from fastapi import Response
        return Response(
            content="frame queue not initialized; arrancar via run_interactive.py",
            status_code=503,
            media_type="text/plain",
        )
    return StreamingResponse(
        _mjpeg_generator(),
        media_type=f"multipart/x-mixed-replace; boundary={_MJPEG_BOUNDARY}",
    )

```